Predicting Credit-Card Default under Class Imbalance with a Class-Balanced Random Forest

Gabriele Adorni
Politecnico di Torino
Data Science and Machine Assignment
Email: s349308@studenti.polito.it

***Abstract*—We address the prediction of next-month credit-card default on the UCI "default of credit card clients" data, a binary classification task whose strong class imbalance (about 22% defaulters) makes raw accuracy misleading; the task is therefore optimised for macro-F1. We build a leakage-free scikit-learn pipeline that folds undocumented categorical codes, imputes and encodes the features, and tunes the decision threshold for macro-F1. A histogram-based gradient-boosting baseline reaches a cross-validated macro-F1 of 0.709 and a public-leaderboard score of 0.712. We then move to a class-balanced random forest and an "additional step" of feature engineering that respects the *semantics* of the repayment-status codes (which are not a monotone scale): keeping the no-consumption, paid-in-full and revolving-credit states distinct gives the forest's axis-aligned splits explicit signal that the gradient-boosting model did not exploit. This lifts the public leaderboard from 0.712 to 0.721. We further report that on this data the development cross-validation is decoupled from the leaderboard, so feature candidates were ranked by the leaderboard rather than by out-of-fold score.**

I. Problem Overview

The task is to predict whether a credit-card client will *default* on their next monthly payment, a binary classification problem ($0 =$ no default, $1 =$ default) on the UCI "default of credit card clients" dataset [1], [2]. The full dataset describes 30,000 clients in Taiwan. For this project it is split into a labelled *development* set of 24,000 clients, used for all training and cross-validation, and an unlabelled *evaluation* set of 6,000 clients whose predictions are scored on the course leaderboard.

Each client is described by 23 features: the credit limit (`LIMIT_BAL`), demographics (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), six months of repayment status (`PAY_0`, `PAY_2`–`PAY_6`), six monthly bill statements (`BILL_AMT1`–`6`) and six monthly payment amounts (`PAY_AMT1`–`6`). `SEX`, `EDUCATION` and `MARRIAGE` are nominal; the remaining features, including the repayment-status codes, are numeric/ordinal.

The data contains documented and undocumented quirks that the preprocessing must handle. `EDUCATION` contains the undocumented codes $\{0, 5, 6\}$ (the specification documents only 1–4) and `MARRIAGE` contains the undocumented code $\{0\}$ (documented 1–3); both are folded into the existing "other" category. The repayment-status columns take values in $\{-2, -1, 0, 1, \ldots, 9\}$, where the specification only documents -1 (paid duly) and 1–9 (months of delay): in practice -2 (no consumption) and 0 (revolving credit) also occur and carry distinct meaning, a point we exploit later.

Class imbalance and metric. The development set is imbalanced: about 77.9% of clients do not default and only 22.1% do. Under this skew, a trivial classifier that always predicts "no default" would already score about 78% accuracy while being useless for the minority class that matters. We therefore optimise **macro-F1** [5], the unweighted mean of the per-class F1 scores, which weights the default and non-default classes equally and rewards correctly recovering the minority (default) class.

II. Proposed Approach

The model is a single scikit-learn [4] `Pipeline` so that every fitted transform sees only the training folds during cross-validation, guaranteeing no information leaks from validation or evaluation data. The pipeline chains code-folding, optional feature engineering and encoding, a column-wise preprocessor, the estimator, and a final macro-F1 threshold tuner.

A. Preprocessing

Cleaning is a *stateless, deterministic* mapping (it fits nothing), so it is inherently leakage-free: the undocumented `EDUCATION` codes $\{0, 5, 6\}$ are folded to 4 and the undocumented `MARRIAGE` code $\{0\}$ to 3, collapsing them into the documented "other" buckets.

The remaining preprocessing is a `ColumnTransformer` with two branches. Numeric features are median-imputed (a guard against unexpected missing values, including any produced by guarded division in engineered features) and optionally standardised. The three nominal features (`SEX`, `EDUCATION`, `MARRIAGE`) are most-frequent-imputed and one-hot encoded, with unknown categories ignored at transform time so the evaluation set never breaks the encoder. The repayment-status columns are kept as ordinal numeric inputs rather than one-hot encoded, since their ordering carries the delinquency signal.

Standardisation is treated as a *per-estimator* switch. A monotone per-feature rescaling such as `StandardScaler` leaves a tree-based model unchanged, because a tree only depends on the order of split points; we verified this is bit-identical for the gradient-boosting model. The scaler is

therefore dropped on the tree path as a cost-free simplification and retained only when a linear model needs it.

B. Model Selection

As a first strong baseline we compare three estimators of increasing capacity under identical preprocessing and cross-validation: a stratified random classifier (a floor), a class-balanced logistic regression (an interpretable linear baseline), and a histogram-based gradient-boosting classifier (HGB). A gradient-boosted decision tree is a natural first choice for this problem: it is a strong off-the-shelf learner for heterogeneous tabular data, captures non-linear interactions among the repayment, bill and payment histories without manual feature crossing, is insensitive to feature scaling and monotone transforms, and handles the mixed numeric/categorical inputs robustly. As shown in Section III, HGB is the best of the three baselines, so it anchors the rest of the study.

Bake-off. Beyond the baseline we compared candidate learners under identical preprocessing with paired repeated cross-validation (Section II-C). Against the HGB anchor, a random forest [3] was the most reliable competitor (it improved macro-F1 on every fold seed), a logistic regression was weaker but the most *decorrelated* from the tree models, and plain gradient boosting and extra-trees were within noise. The single-model gains were small, but the random forest's robustness made it the platform for the rest of the study.

Handling the imbalance. On a $78/22$ split, a classifier trained to minimise error under-weights the minority (default) class. We address this at two points: `class_weight="balanced"` re-weights the forest's split criterion inversely to class frequency, and the macro-F1 threshold tuner (Section II-C) sets the operating point. This class-balanced random forest (300 trees, `min_samples_leaf`=20) improves the public leaderboard from 0.712 to 0.713 over the gradient-boosting baseline; in cross-validation it beats HGB on every fold seed.

The decisive step: repayment-status semantics. The largest gain came from feature engineering that respects what the repayment-status codes *mean*. The codes $\{-2, -1, 0, 1, \ldots, 9\}$ are **not** a monotone scale: -2 is no consumption, -1 is the balance paid in full, 0 is revolving credit (a balance carried but not yet delinquent, a genuinely riskier state), and 1–9 count months of delay. Treating the column as a single ordinal number, or collapsing $\{-2, -1, 0\}$ into one "not late" bucket, discards the revolving-versus-paid-in-full distinction. Our `paysem` feature family makes these states explicit: per client it counts the months spent revolving, paid-in-full and at no-consumption, and flags the latest month's state (revolving, paid-in-full, or two-or-more months late). We pair this with `payratio` (payment-coverage ratios: fraction of each bill paid, coverage of the previous month's bill, counts of zero and full payments) and `stress`, which forms explicit delinquency$\times$utilisation interactions (e.g. being late *and* near the credit limit). All engineered features are row-wise, stateless functions of a single client's own columns, so they are computed identically on the development and evaluation sets and add no leakage.

Crucially, this engineering helped the *random forest* but not the gradient-boosting model. A forest splits on raw axis-aligned thresholds, so an explicit ratio or a semantic count is a feature it can split on directly; the histogram-based booster bins each feature monotonically and already absorbs such monotone re-derivations, so the same features were flat for it. On the random forest the engineered families lifted the public leaderboard to 0.716 (`+payratio`) and then to 0.720–0.721 once the `paysem` semantics were added. The **deployed model** is therefore a class-balanced random forest on the `paysem+stress+payratio` feature set with macro-F1 threshold tuning; it is defined once in `config.yaml` (the `chosen` block) and is exactly what `main.py` retrains and submits.

C. Hyperparameter Tuning

All model selection and tuning is performed with stratified 5-fold cross-validation (`StratifiedKFold`, shuffled, fixed seed) on the development set, and is kept out of the submission entry point: `main.py` only trains the single chosen configuration. To separate a real edge from a lucky split, candidates are compared with *paired repeated* cross-validation: the same five fold seeds are used for both models, and a candidate is preferred only when it improves the out-of-fold macro-F1 on the mean *and* wins on every seed.

Deployed objective. Because the final classifier submits behind a tuned decision threshold, we do not score candidates at the default 0.5 cut. Instead, for each fold we obtain out-of-fold class probabilities and select the threshold that maximises macro-F1 over a grid of 181 values spanning 0.05 to 0.95. This *threshold-at-best-macro-F1* on out-of-fold probabilities is the objective every experiment reports, matching what the deployed model does. In production the threshold is set by `TunedThresholdClassifierCV`, which chooses it on inner cross-validation folds only, so the operating point is selected without leakage.

Threshold tuning is itself a sizeable, free gain: it lifts the baseline HGB macro-F1 from 0.688 at the 0.5 cut to 0.709 at the tuned threshold (≈ 0.33), an improvement of about $+0.021$ obtained without changing the model. A randomised search over the HGB hyperparameters (learning rate, number of iterations, tree size, regularisation) yielded only a marginal cross-validated gain ($\leq +0.002$ macro-F1) that did not justify departing from the library defaults; the substantial improvements came instead from the model family and feature engineering of Section II-B.

Cross-validation is decoupled from the leaderboard. A consistent finding on this dataset is that out-of-fold macro-F1 does *not* rank the candidates the way the leaderboard does: the differences between strong random-forest feature sets are smaller than the cross-validation's own seed-to-seed wobble, and the best-leaderboard configuration is not the best out-of-fold (Table II). We therefore used cross-validation as a guard

TABLE I
BASELINE COMPARISON (5-FOLD CV, MACRO-F1 AT EACH MODEL'S TUNED THRESHOLD). HGB IS THE STRONGEST BASELINE; ITS PUBLIC LEADERBOARD SCORE IS 0.712.

Model	CV macro-F1	ROC-AUC
Stratified dummy	0.505	0.505
Logistic regression (balanced)	0.697	0.728
HistGradientBoosting	**0.709**	**0.782**

TABLE II
MILESTONE PROGRESSION (CV MACRO-F1 OUT-OF-FOLD AT THE DEPLOYED SEED). THE RANDOM-FOREST CV SCORES ARE FLAT TO WITHIN ± 0.001 (SEED-TO-SEED NOISE), YET THE PUBLIC LEADERBOARD (LB) RISES BY 0.008; ONLY THE LB SEPARATES THE FEATURE SETS. PS=PAYSEM, UT=UTIL, ST=STRESS, PR=PAYRATIO.

Configuration	CV macro-F1	Public LB
HGB baseline	0.709	0.712
Random forest (`balanced`)	0.711	0.713
+ `payratio`	0.711	0.716
+ `ps+ut+pr` (pick A)	0.711	0.720
+ `ps+st+pr` (pick B, deployed)	0.711	**0.721**

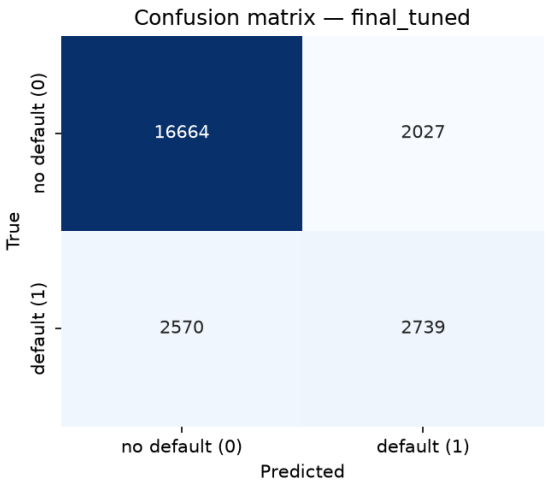

Fig. 1. Deployed model confusion matrix (out-of-fold, tuned threshold ≈ 0.61) on the 24,000-client development set: class-balanced random forest on the `paysem+stress+payratio` features.

against clearly worse models, but *ranked* the close feature-engineering candidates on the random forest by their public-leaderboard score rather than by out-of-fold macro-F1.

III. RESULTS

Table I reports the three baselines under the deployed objective: 5-fold cross-validated macro-F1 at each model's own tuned threshold, plus ROC-AUC. The gradient-boosting model is clearly the strongest baseline, and its cross-validated macro-F1 of 0.709 corresponds to a public-leaderboard score of **0.712**.

From baseline to deployed model. Table II traces the public-leaderboard progression alongside the development cross-validation score of each milestone. Two things stand out. First, moving from the gradient-boosting baseline to the class-balanced random forest, and then adding the repayment-status feature families, lifts the leaderboard from 0.712 to 0.721. Second, across the random-forest rows the cross-validation column is essentially flat (within ± 0.001) while the leaderboard moves by 0.008, and the best-leaderboard configuration is not the best out-of-fold: development cross-validation does not predict the leaderboard ordering here, which is why the close candidates were ranked by the leaderboard (Section II-C).

Final model. The deployed configuration (pick B: class-balanced random forest on `paysem+stress+payratio`, macro-F1 threshold tuning) reaches an out-of-fold macro-F1 of 0.711 at a tuned threshold of ≈ 0.61 and a public-leaderboard score of 0.721. Its confusion matrix at this operating point (Fig. 1) gives per-class precision/recall/F1 of $0.866/0.892/0.879$ for non-default and $0.575/0.516/0.544$ for default: the threshold deliberately trades a little non-default accuracy to recover about half of the true defaulters, which is what macro-F1 rewards. As the rules allow two final submissions and only the private leaderboard counts, we keep pick B (deployed, 0.721) and pick A (`paysem+util+payratio`, 0.720) as the two finalists; A and B differ in their third feature family and make partly different errors, hedging the private-leaderboard draw.

IV. DISCUSSION

Error analysis. The residual error is concentrated on the minority class: at the deployed operating point the model recovers about 52% of true defaulters (recall 0.516) at precision 0.575, so roughly half of all defaults are still missed and a comparable number of non-defaulters are flagged. This is the expected shape under macro-F1 – the tuned threshold (≈ 0.61) deliberately moves off the probability midpoint to balance the two per-class F1 scores rather than to maximise accuracy. The hard cases are clients with mixed signals (some late months but recent recovery, or high utilisation while still paying), where six months of history do not settle the outcome.

Cross-validation versus leaderboard. The most important methodological lesson is that, beyond separating clearly different models, development cross-validation did not track the public leaderboard on this data: competitive feature sets differed by less than the cross-validation's seed-to-seed wobble, and the best out-of-fold candidate was not the best on the leaderboard. We handled this by treating cross-validation as a coarse filter and using the leaderboard to rank the close candidates, and by carrying *two* diverse finalists (picks A and B) into the private leaderboard rather than over-fitting to a single public score.

Random forest versus gradient boosting. The same engineered features that lifted the random forest were inert for the gradient-boosting baseline. A histogram booster bins each feature and learns monotone responses, so a re-derivation such as a utilisation ratio or a count of revolving months adds little it cannot already recover; a random forest splits on raw thresholds, so an explicit ratio or semantic count is directly

actionable. This is why the decisive gain came from pairing the `paysem` semantics *with* the random forest, not from a stronger single learner.

What did not help. Several alternatives were screened and rejected on this data: a gradient-boosted alternative (LightGBM) only matched the random forest and did not justify an extra dependency; resampling the training set (SMOTE and variants) underperformed plain class weighting; and soft-voting ensembles that looked promising in cross-validation did not transfer to the leaderboard. For an imbalanced tabular task with a thresholded metric, class weighting plus threshold tuning was both simpler and stronger.

Limitations. Tree methods plateaued near 0.72 on the public leaderboard across many feature and ensemble variants, suggesting the remaining signal in six months of history is limited; the assignment also forbids external data. All methods used (random forests, gradient boosting, logistic regression, class weighting and threshold tuning) are standard supervised-learning techniques within the course syllabus, so no out-of-syllabus justification is required.

REFERENCES

[1] I.-C. Yeh and C.-H. Lien, "The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2473–2480, 2009.

[2] UCI Machine Learning Repository, "Default of credit card clients data set," https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.